

# AP CSP Vocabulary Glossary

Working in Python · alphabetical · 300 terms · all terms

Terms marked <sup>AP</sup> appear on the College Board AP CSP exam's own vocabulary list. A superscript chapter number links to where this book defines the term.

Vocabulary drawn from *Working in Python*, a fork of Allen Downey's *Think Python*, 3rd edition (CC BY-NC-SA 4.0), and this course's own AP CSP vocabulary reference. AP CSP definitions here are written in this course's own words, not copied from the College Board's Course and Exam Description.

## A

**absolute path** <sup>13</sup> A path that does not depend on the current directory.

**abstraction** <sup>AP</sup> Reducing complexity by exposing essential features and hiding implementation detail.

**accumulator** <sup>10</sup> A variable used in a loop to add up or accumulate a result.

**aggregation of information** <sup>AP</sup> Combining separately harmless pieces of data to identify or profile a person.

**algorithm** <sup>AP</sup> A finite sequence of steps that solves a problem or completes a task. Can be written in English, pseudocode, or code.

**algorithmic efficiency** <sup>AP</sup> An estimate of the computational resources an algorithm uses as a function of input size.

**aliased** <sup>9</sup> If there is more than one variable that refers to an object, the object is aliased.

**analog data** <sup>AP 7b</sup> Values that vary continuously and smoothly, with no steps.

**anonymization** <sup>AP</sup> Removing identifying details from data — and its limits, since aggregation can re-identify.

**API** <sup>AP</sup> The specification of how a library's procedures are called and how they behave.

**argument** <sup>AP 2</sup> A value provided to a function when the function is called.

**arithmetic operator** <sup>1</sup> A symbol, like + and \*, that denotes an arithmetic operation like addition or multiplication.

**ASCII** <sup>AP 7b</sup> A table assigning a number from 0 to 127 to each of a small set of characters.

**assignment statement** <sup>AP 2</sup> A statement that assigns a value to a variable.

**attribute** <sup>9,14</sup> A named value associated with an object; called an instance variable when it belongs to a class instance.

**authentication** <sup>AP</sup> Verifying that a user is who they claim to be.

## B

**bandwidth** <sup>AP</sup> Maximum data transmittable per unit time, in bits per second.

**base case** <sup>5</sup> A conditional branch in a recursive function that does not make a recursive call.

**beneficial and harmful effects** <sup>AP</sup> Every computing innovation has both, and they can fall on different groups of people.

**bias** <sup>AP</sup> Systematic unfairness embedded in an algorithm or in the data it was built from.

**bigram** <sup>12</sup> A sequence of two elements, often words.

**binary** <sup>AP 7b</sup> Base-2 representation, using only the digits 0 and 1.

**binary mode** <sup>13</sup> A way of opening a file so the contents are interpreted as a sequence of bytes rather than a sequence of characters.

**binary search** <sup>AP</sup> Repeatedly halving a sorted list to locate a value.

**bit** <sup>AP 7b</sup> A single binary digit, 0 or 1.

**block** <sup>5</sup> One or more statements indented to indicate they are part of another statement.

**body** <sup>3</sup> The sequence of statements inside a function definition.

**boolean expression** <sup>AP 5</sup> An expression whose value is either True or False.

**boundary case** <sup>6b</sup> A test case at the value where a function's behavior changes, such as zero, an empty string, or the first or last item.

**branch** <sup>5</sup> One of the alternative sequences of statements in a conditional statement.

**bug** <sup>1</sup> An error in a program.

**byte** <sup>AP 7b</sup> Eight bits. Enough to hold one of 256 values.

## C

**call graph** <sup>10</sup> A diagram that shows every frame created during the execution of a program, with an arrow from each caller to each callee.

**canvas** <sup>4</sup> A window used to display graphical elements including lines, circles, rectangles, and other shapes.

**certificate authority** <sup>AP</sup> An organization that issues and vouches for digital certificates.

**chained conditional** <sup>5</sup> A conditional statement with a series of alternative branches.

**character** <sup>8</sup> An element of a string, including letters, numbers, and symbols.

**character encoding**<sup>7b</sup> The agreement about which numbers stand for which characters. Read bits with the wrong encoding and you get the right data as the wrong text.

**child class**<sup>17</sup> A class that inherits from another class.

**citizen science**<sup>AP</sup> Scientific research carried out partly by volunteers using their own devices.

**class**<sup>14</sup> A programmer-defined type. A class definition creates a new class object.

**class object**<sup>14</sup> An object that represents a class — it is the result of a class definition.

**class variable**<sup>17</sup> A variable defined inside a class definition, but not inside any method.

**code segment**<sup>AP</sup> A portion of a program, one or more lines, treated as a unit.

**collaboration**<sup>AP</sup> Working with others to develop a computing innovation; benefits from varied perspectives and skills.

**comment**<sup>AP 2</sup> Text included in a program that provides information about the program but has no effect on its execution.

**compound conditional**<sup>AP</sup> A single condition combining multiple Boolean expressions with AND, OR, or NOT.

**compression ratio**<sup>7b</sup> Compressed size divided by original size.

**computer network**<sup>AP</sup> Interconnected computing devices able to send and receive data.

**computer virus**<sup>AP</sup> Malware that copies itself, typically by attaching to other programs or files.

**computing device**<sup>AP</sup> A physical artifact that can run a program.

**computing innovation**<sup>AP</sup> A product, service, or concept that includes a program as an integral part of its function. May be physical, software, or conceptual.

**computing system**<sup>AP</sup> A group of computing devices and programs working together.

**concatenation**<sup>AP 1</sup> Joining two strings end-to-end.

**condition**<sup>5</sup> The boolean expression in a conditional statement that determines which branch runs.

**conditional expression**<sup>18</sup> An expression that uses a conditional to select one of two values.

**conditional statement**<sup>AP 5</sup> A statement that controls the flow of execution depending on some condition. (*AP calls this selection.*)

**configuration data**<sup>13</sup> Data, often stored in a file, that specifies what a program should do and how.

**constant**<sup>AP</sup> A named value that does not change during execution.

**cookies**<sup>AP</sup> Small data files stored by a site to track a user's state or behavior.

**copyright**<sup>AP</sup> The exclusive right of a creator to reproduce, distribute, and display a work.

**correlation**<sup>AP</sup> An association between two variables where changes in one accompany changes in the other.

**counter**<sup>7</sup> A variable used to count something, usually initialized to zero and then incremented.

**creative commons**<sup>AP</sup> A set of public licenses letting a creator pre-authorize specific reuse under stated conditions.

**crowdsourcing**<sup>AP</sup> Gathering input, funding, or labor from many people over the Internet.

**current working directory**<sup>13</sup> The default directory used by a program unless another directory is specified.

## D

**data**<sup>AP</sup> Values that can be stored and processed by a program.

**data abstraction**<sup>AP</sup> Using a data structure, typically a list, to represent related values as one named unit, hiding the individual pieces.

**data cleaning**<sup>AP</sup> Making data uniform and consistent without changing its meaning.

**data mining**<sup>AP</sup> Analyzing large data sets to find patterns and relationships.

**data set**<sup>AP</sup> A collection of related data organized for analysis.

**data structure**<sup>11</sup> A collection of values, organized to perform certain operations efficiently.

**database**<sup>13</sup> A file whose contents are organized to perform certain operations efficiently.

**dead code**<sup>6</sup> Part of a program that can never run, often because it appears after a return statement.

**debugging**<sup>AP 1</sup> The process of finding and correcting errors.

**decidable problem**<sup>AP</sup> A yes/no problem for which an algorithm can always produce the correct answer.

**decimal**<sup>AP 7b</sup> Base-10, the system you already use.

**decrement**<sup>7</sup> Decrease the value of a variable.

**deep copy**<sup>16</sup> A copy operation that also copies nested objects.

**default value**<sup>12</sup> The value assigned to a parameter if no argument is provided.

**delegation**<sup>17</sup> When one method passes responsibility to another method to do most or all of the work.

**delimiter**<sup>9</sup> A character or string used to indicate where a string should be split.

**deserialization**<sup>13</sup> Converting a string to an object.

**design-first development**<sup>14</sup> A way of developing programs with more careful planning than prototype and patch.

**deterministic**<sup>12</sup> A deterministic program does the same thing each time it runs, given the same inputs.

**development plan**<sup>4</sup> A process for writing programs.

**dictionary**<sup>10</sup> An object that contains key-value pairs, also called items.

**digest**<sup>13</sup> The result of a hash function, especially when it is used to check whether two objects are the same.

**digital certificate**<sup>AP</sup> A credential issued by an authority binding a public key to an identity.

**digital data**<sup>AP 7b</sup> Values represented in discrete steps, ultimately as bits.

**digital divide**<sup>AP</sup> Unequal access to computing and the Internet across socioeconomic, geographic, or demographic lines.

**directory**<sup>13</sup> A collection of files and other directories.

**distributed computing**<sup>AP</sup> Multiple devices in different locations running parts of one program.

**DNS**<sup>AP</sup> Translates domain names into IP addresses.

**docstring**<sup>AP 4,6b</sup> A string at the beginning of a function that documents what the function does; unlike a comment, it is stored on the function and can be read by the program. (*AP calls this program documentation.*)

**doctest**<sup>6b</sup> An example call and its expected result, written inside a docstring, that can be run automatically to check the function.

**dot operator**<sup>2</sup> The operator, `.`, used to access a function in another module by specifying the module name followed by a dot and the function name.

## E

**edge case**<sup>6b</sup> A test case at an unusual or extreme input, where a function is most likely to be wrong.

**element**<sup>AP 9</sup> One of the values in a list or other sequence.

**empty string**<sup>8</sup> A string that contains no characters and has length 0.

**encapsulation**<sup>4</sup> The process of transforming a sequence of statements into a function definition.

**encode**<sup>17</sup> To represent one set of values using another set of values by constructing a mapping between them.

**encryption**<sup>AP</sup> Encoding data so only holders of the key can read it.

**enumerate object**<sup>11</sup> The result of calling the built-in function `enumerate`, can be used to loop through a sequence of tuples.

**ephemeral**<sup>13</sup> An ephemeral program typically runs for a short time and, when it ends, its data are lost.

**equivalent**<sup>9</sup> Having the same value.

**evaluate**<sup>2</sup> Perform the operations in an expression in order to compute a value.

**event**<sup>AP</sup> An action supplied to a program as input: key press, click, sensor reading.

**event-driven programming**<sup>AP</sup> Statements execute in response to events rather than in top-to-bottom sequence.

**exception**<sup>2</sup> An error that is detected while the program is running.

**execute**<sup>2</sup> Run a statement and do what it says.

**expected value**<sup>6b</sup> What a test says the answer should be, as opposed to what the code actually produced.

**expression**<sup>AP 1</sup> A combination of variables, values, and operators.

## F

**f-string**<sup>13</sup> A string that has the letter `f` before the opening quotation mark, and contains one or more expressions in curly braces.

**factory**<sup>18</sup> A function used to create objects, often passed as a parameter to a function.

**fail**<sup>6b,7</sup> A test whose actual result does not match its expected value.

**fault tolerance**<sup>AP</sup> Continuing to operate when components fail.

**file object**<sup>7</sup> An object that represents an open file and keeps track of which parts of the file have been read or written.

**filtering**<sup>AP 10</sup> Looping through a sequence and selecting or omitting elements. (*AP calls this data filtering.*)

**floating-point**<sup>1</sup> A type that represents integers and numbers with decimal parts.

**formal language**<sup>1</sup> Any of the languages that people have designed for specific purposes, such as representing mathematical ideas or computer programs. All programming languages are formal languages.

**format specifier**<sup>14</sup> In an f-string, a format specifier determines how a value is converted to a string.

**frame**<sup>3</sup> A box in a stack diagram that represents a function call. It contains the local variables and parameters of the function.

**function**<sup>AP 1</sup> A named sequence of statements that performs some useful operation. Functions may or may not take arguments and may or may not produce a result. (*AP calls this a procedure.*)

**function call**<sup>1</sup> An expression — or part of an expression — that runs a function. It consists of the function name followed by an argument list in parentheses.

**function definition**<sup>3</sup> A statement that creates a function.

**function object**<sup>3</sup> A value created by a function definition. The name of the function is a variable that refers to a function object.

**functional programming style**<sup>14</sup> A way of programming that uses pure functions whenever possible.

## G

**generalization**<sup>4</sup> The process of replacing something unnecessarily specific (like a number) with something appropriately general (like a variable or parameter).

**generator expression**<sup>18</sup> Similar to a list comprehension except that it does not create a list.

## H

**hand tracing**<sup>AP 6b</sup> Working through code on paper line by line, writing down each variable's value, in order to find an error without running the program.

**hash function**<sup>10,13</sup> A function that takes an object and computes an integer — sometimes called a digest — used to locate a key in a hash table or to check whether two objects are the same.

**hash table**<sup>10</sup> A collection of key-value pairs organized so that we can look up a key and find its value efficiently.

**hashable**<sup>10</sup> Immutable types like integers, floats and strings are hashable. Mutable types like lists and dictionaries are not.

**header**<sup>3</sup> The first line of a function definition.

**heuristic**<sup>AP</sup> An approach that finds a good-enough solution when finding the optimal one is impractical.

**hexadecimal**<sup>AP 7b</sup> Base-16, using 0 through 9 and A through F. Four bits per digit, so one byte is exactly two hex digits.

**HTTP / HTTPS**<sup>AP</sup> The Web's request/response protocol; HTTPS adds encryption in transit.

## I

**identical**<sup>9</sup> Being the same object (which implies equivalence).

**immutable**<sup>8</sup> If the elements of an object cannot be changed, the object is immutable.

**import statement**<sup>2</sup> A statement that reads a module file so we can use the variables and functions it contains.

**increment**<sup>7</sup> Increase the value of a variable.

**incremental development**<sup>AP 6</sup> A program development plan intended to avoid debugging by adding and testing only a small amount of code at a time.

**index**<sup>AP 8</sup> An integer value used to select an item in a sequence, such as a character in a string. In Python indices start from 0.

**infinite loop**<sup>AP</sup> A loop whose ending condition never becomes true.

**infinite recursion**<sup>5</sup> A recursion that doesn't have a base case, or never reaches it. Eventually, an infinite recursion causes a runtime error.

**information**<sup>AP</sup> Meaning, patterns, or insight extracted from data.

**inheritance**<sup>17</sup> The ability to define a new class that is a modified version of a previously defined class.

**initialize**<sup>7</sup> Create a new variable and give it a value.

**input validation**<sup>6</sup> Checking the parameters of a function to make sure they have the correct types and values.

**instance**<sup>14</sup> An object that belongs to a class.

**instance method**<sup>15</sup> A method that must be invoked with an object as receiver.

**instantiation**<sup>14</sup> The process of creating an object that belongs to a class.

**integer**<sup>AP 1</sup> A type that represents numbers with no fractional or decimal part.

**integer division**<sup>1</sup> An operator, //, that divides two numbers and rounds down to an integer.

**intellectual property**<sup>AP</sup> Creative or technical work legally owned by its creator.

**interface design**<sup>AP 4</sup> A process for designing the interface of a function, which includes the parameters it should take. (*AP calls this procedural abstraction.*)

**internet**<sup>AP</sup> A network of interconnected networks using standardized open protocols.

**invariant**<sup>15</sup> A condition that should always be true during the execution of a program.

**invocation**<sup>8</sup> An expression — or part of an expression — that calls a method.

**IP**<sup>AP</sup> Protocol for addressing devices and routing packets between them.

**IP address**<sup>AP</sup> The unique identifier assigned to a device on a network.

**item**<sup>10</sup> In a dictionary, another name for a key-value pair.

**iterative development**<sup>AP</sup> Repeated cycles of build, feedback, and revision; earlier phases get revisited.

## K

**key**<sup>10</sup> An object that appears in a dictionary as the first part of a key-value pair.

**key-value stores**<sup>13</sup> A database whose contents are organized like a dictionary with keys that correspond to values.

**keylogging**<sup>AP</sup> Recording keystrokes to capture passwords and confidential input.

**keyword**<sup>2</sup> A special word used to specify the structure of a program.

**keyword argument**<sup>4</sup> An argument that includes the name of the parameter.

## L

**linear search**<sup>AP 7</sup> A computational pattern that searches through a sequence of elements and stops when it finds what it is looking for.

**list**<sup>AP 9</sup> An object that contains a sequence of values. (*on the AP exam, list indices start at 1; in Python, at 0.*)

**list comprehension**<sup>18</sup> A concise way to loop through a sequence and create a list.

**local variable**<sup>3</sup> A variable defined inside a function, and which can only be accessed inside the function.

**logical operator**<sup>AP 5</sup> One of the operators that combines boolean expressions, including and, or, and not.

**loop**<sup>AP 3</sup> A statement that runs one or more statements, often repeatedly. (*AP calls this iteration.*)

**loop variable**<sup>7</sup> A variable defined in the header of a for loop.

**lossless compression**<sup>AP 7b</sup> Reduces size while allowing the original to be reconstructed exactly.

**lossy compression**<sup>AP 7b</sup> Reduces size further, but only an approximation of the original can be recovered.

## M

**machine learning**<sup>AP</sup> Systems that improve at a task from data rather than from explicit programming.

**malware**<sup>AP</sup> Software intended to damage or take control of a system.

**mapping**<sup>10</sup> A relationship in which each element of one set corresponds to an element of another set.

**memo**<sup>10</sup> A computed value stored to avoid unnecessary future computation.

**metadata**<sup>AP</sup> Data describing other data: creation date, size, author, location, format.

**method**<sup>7,15</sup> A function that is associated with an object and called using the dot operator; when defined inside a class it is invoked on instances of that class.

**modularity**<sup>AP</sup> Dividing a program into independent parts each responsible for one aspect.

**module**<sup>AP 2</sup> A file that contains Python code, including function definitions and sometimes other statements. (*AP calls this a software library.*)

**modulus operator**<sup>AP 5</sup> An operator, %, that works on integers and returns the remainder when one number is divided by another. (*AP calls this MOD.*)

**multifactor authentication**<sup>AP</sup> Requiring evidence from two or more categories: knowledge, possession, inherence.

**multiline string**<sup>4</sup> A string enclosed in triple quotes that can span more than one line of a program.

## N

**n-gram**<sup>12</sup> A sequence of an unspecified number of elements.

**natural language**<sup>1</sup> Any of the languages that people speak that evolved naturally.

**nested conditional**<sup>AP 5</sup> A conditional statement that appears in one of the branches of another conditional statement.

**nested list**<sup>9</sup> A list that is an element of another list.

**newline**<sup>5</sup> A character that creates a line break between two parts of a string.

## O

**object**<sup>8</sup> Something a variable can refer to. An object has a type and a value.

**object diagram**<sup>14</sup> A graphical representation of an object, its attributes, and their values.

**object-oriented language**<sup>15</sup> A language that provides features to support object-oriented programming, notably user-defined types.

**object-oriented programming**<sup>14</sup> A style of programming that uses objects to organize code and data.

**open access**<sup>AP</sup> Research or data made available free of access restrictions.

**open source**<sup>AP</sup> Software whose source is public and may be modified and redistributed.

**open standard**<sup>AP</sup> A publicly available specification that anyone may implement without permission or fee.

**operand**<sup>1</sup> One of the values on which an operator operates.

**operator overloading**<sup>15</sup> The process of using special methods to change the way operators work with user-defined types.

**overflow error**<sup>AP 7b</sup> An error that happens when a value is too large for the number of bits available to hold it.

**override**<sup>12</sup> To replace a default value with an argument.

## P

**pack**<sup>11</sup> Collect multiple arguments into a tuple.

**packet**<sup>AP</sup> A chunk of data carrying metadata such as its destination, sent across a network.

**packet switching**<sup>AP</sup> Splitting a message into packets that travel independently, possibly by different routes, and are reassembled at the destination.

**pair programming**<sup>AP</sup> Two programmers at one workstation: one drives, one reviews and thinks ahead.

**parallel computing**<sup>AP</sup> A program split into operations, some performed simultaneously.

**parameter**<sup>AP 3</sup> A name used inside a function to refer to the value passed as an argument.

**parent class**<sup>17</sup> A class that is inherited from.

**pass**<sup>6b,7</sup> A test whose actual result matches its expected value.

**path**<sup>13</sup> A string that specifies a sequence of directories, often leading to a file.

**pattern**<sup>8</sup> A rule that specifies the requirements a string has to meet to constitute a match.

**persistent**<sup>13</sup> A persistent program runs indefinitely and keeps at least some of its data in permanent storage.

**phishing**<sup>AP</sup> Tricking a user into revealing private information by impersonating a trusted party.

**PII**<sup>AP</sup> Information that identifies or can be linked to a specific individual.

**pixel**<sup>AP</sup> The smallest addressable color element of a display or image.

**plagiarism**<sup>AP</sup> Presenting someone else's work as your own.

**polymorphism**<sup>16</sup> The ability of a method or operator to work with multiple types of objects.

**postcondition**<sup>4</sup> A requirement that should be satisfied by the function before it ends.

**precondition**<sup>4</sup> A requirement that should be satisfied by the caller before a function starts.

**procedure**<sup>AP 6b</sup> The exam's word for a named, reusable block of code, whether or not it returns a value. This book says function. In older languages the two words were distinct: a procedure returned nothing, a function returned a value.

**program**<sup>AP</sup> A collection of statements that performs a task when run.

**program function**<sup>AP 6b</sup> What a program does when it runs, described as behavior.

**program input**<sup>AP 6b</sup> Data a program receives while it is running.

**program output**<sup>AP 6b</sup> What a program produces: displayed text, a returned value, a file, a sound, a movement.

**program purpose**<sup>AP 6b</sup> The need a program serves, or the problem it solves; why it exists.

**protocol**<sup>AP</sup> An agreed set of rules governing system behavior.

**prototype and patch**<sup>14</sup> A way of developing programs by starting with a rough draft and gradually adding features and fixing bugs.

**pseudorandom**<sup>AP 12</sup> A pseudorandom sequence of numbers appears to be random, but is generated by a deterministic program. (*AP calls this `RANDOM(a, b)` — inclusive on both ends, unlike Python's `range()`.*)

**public key encryption**<sup>AP</sup> A public key encrypts; a different private key decrypts.

**pure function**<sup>6,14</sup> A function that does not modify its parameters or have any effect other than returning a value.

## R

**reasonable time**<sup>AP</sup> Run time that grows polynomially or slower with input size.

**receiver**<sup>15</sup> The object a method is invoked on.

**recursion**<sup>5</sup> The process of calling the function that is currently executing.

**recursive**<sup>5</sup> A function that calls itself is recursive.

**redundancy**<sup>AP</sup> Duplicate paths or components that allow operation to continue after a failure.

**refactoring**<sup>4</sup> The process of modifying a working program to improve function interfaces and other qualities of the code.

**reference**<sup>9</sup> The association between a variable and its value.

**regression**<sup>6b</sup> A bug that reappears in code that used to work.

**regular expression**<sup>8</sup> A sequence of characters that defines a search pattern.

**relational operator**<sup>AP 5</sup> One of the operators that compares its operands: `==`, `!=`, `>`, `<`, `>=`, and `<=`.

**relative path**<sup>13</sup> A path that starts from the current working directory, or some other specified directory.

**return value**<sup>AP 6</sup> The result of a function. If a function call is used as an expression, the return value is the value of the expression. (*AP calls this the `RETURN` statement.*)

**RGB**<sup>AP 7b</sup> Color stored as three numbers, the amounts of red, green, and blue, each usually one byte.

**rogue access point**<sup>AP</sup> An unauthorized wireless access point used to intercept network traffic.

**roundoff error**<sup>AP 6b,7b</sup> A loss of precision that happens because a fixed number of bits cannot represent some numbers exactly.

**router**<sup>AP</sup> A device that forwards packets between networks toward their destination.

**routing**<sup>AP</sup> Determining a path from sender to receiver across a network.

**rubber duck debugging**<sup>12</sup> A way of debugging by explaining a problem aloud to an inanimate object.

**run-length encoding**<sup>7b</sup> A lossless scheme that replaces runs of a repeated value with the value and a count.

**runtime error**<sup>AP 2</sup> An error that causes a program to display an error message and exit.

## S

**sampling**<sup>AP 7b</sup> Approximating an analog signal by measuring it at regular intervals. Two independent settings: how often you measure (rate) and how precisely you record each measurement (bit depth).

**scaffolding**<sup>6</sup> Code that is used during program development but is not part of the final version.

**scalability**<sup>AP</sup> A system's ability to keep working acceptably as demand grows.

**semantic error**<sup>AP 2,6b</sup> An error that lets the program run but produces a wrong result. (*AP calls this a `logic error`.*)

**sequence**<sup>8</sup> An ordered collection of values where each value is identified by an integer index.

**sequencing**<sup>AP</sup> Executing steps in the order written.

**sequential computing**<sup>AP</sup> Operations performed one at a time, in order.

**serialization**<sup>13</sup> Converting an object to a string.

**shallow copy**<sup>16</sup> A copy operation that does not copy nested objects.

**shell command**<sup>8</sup> A statement in a shell language, which is a language used to interact with an operating system.

**side effect**<sup>6</sup> Any effect a function has other than returning a value, such as displaying output or drawing on a canvas.

**simulation**<sup>AP</sup> An abstraction of a real phenomenon using varying values to represent changing states, run for a purpose.

**slice**<sup>8</sup> A part of a string specified by a range of indices.

**sort key**<sup>11</sup> A value, or function that computes a value, used to sort the elements of a collection.

**special method**<sup>15</sup> A method that changes the way operators and some functions work with an object.

**specialization**<sup>17</sup> A way of using inheritance to create a new class that is a specialized version of an existing class.

**speedup**<sup>AP</sup> Sequential time divided by parallel time.

**stack diagram**<sup>3</sup> A graphical representation of a stack of functions, their variables, and the values they refer to.

**state diagram**<sup>2</sup> A graphical representation of a set of variables and the values they refer to.

**statement**<sup>2</sup> One or more lines of code that represent a command or action.

**static method**<sup>15</sup> A method that can be invoked without an object as receiver.

**string**<sup>AP 1</sup> A type that represents sequences of characters.

**string substitution**<sup>8</sup> Replacement of a string, or part of a string, with another string.

**substring**<sup>AP</sup> A contiguous portion of a string.

**symmetric key encryption**<sup>AP</sup> One shared key both encrypts and decrypts.

**syntax error**<sup>AP 1</sup> An error in a program that makes it impossible to parse — and therefore impossible to run.

## T

**TCP**<sup>AP</sup> Transport protocol providing reliable, ordered delivery with retransmission.

**test case**<sup>AP 6b</sup> A specific input paired with its expected output. Good sets include typical, boundary, and edge values.

**test discovery**<sup>18</sup> A process used to find and run tests.

**testing**<sup>AP 6b</sup> Checking that a program behaves correctly by running it on chosen inputs and comparing what comes out against what should have come out.

**totally ordered**<sup>17</sup> A set of objects is totally ordered if we can compare any two elements and the results are consistent.

**traceback**<sup>3</sup> A list of the functions that are executing, printed when an exception occurs.

**traversal**<sup>AP</sup> Iterating over the items of a list. Full traversal visits every element; partial stops early.

**trigram**<sup>12</sup> A sequence of three elements.

**Turing complete**<sup>6</sup> A language, or subset of a language, is Turing complete if it can perform any computation that can be described by an algorithm.

**type**<sup>AP 1</sup> A category of values. The types we have seen so far are integers (type int), floating-point numbers (type float), and strings (type str). (*AP calls this data type.*)

## U

**UDP**<sup>AP</sup> Lightweight transport protocol with minimal error checking; faster, less reliable.

**undecidable problem**<sup>AP</sup> A problem for which no algorithm can always give a correct yes/no answer for every input.

**Unicode**<sup>AP 7b</sup> A far larger table, covering the writing systems ASCII left out.

**unpack**<sup>11</sup> Treat a tuple (or other sequence) as multiple arguments.

**unreasonable time**<sup>AP</sup> Run time that grows exponentially or factorially with input size.

**update**<sup>7</sup> An assignment statement that gives a new value to a variable that already exists, rather than creating a new variable.

## V

**value**<sup>1,10</sup> An integer, floating-point number, string, or other kind of data a program works with. In a dictionary, a value is specifically the second part of a key-value pair.

**variable**<sup>AP 2</sup> A name that refers to a value.

## W

**world wide web**<sup>AP</sup> A system of linked pages and files that runs on top of the Internet using HTTP.

## Z

**zip object**<sup>11</sup> The result of calling the built-in function zip, can be used to loop through a sequence of tuples.